IC Design Laboratory
EE - 652
Group no :- 5

Module Description

1. Instruction Fetch (IF) Cycle

Name: Saksham Pandey
Roll Number: 25210111
Assigned Module: Instruction Fetch (IF) Cycle

Module Description:
It is the first stage and it generates the next instruction address, fetches the instruction from instruction memory, and supplies it to the next pipeline stage.
It consists of Program Counter, Instruction Memory and IF modules integrated to perform desired operation.
The IF stage performs the following operations every clock cycle:

1. Maintains the Program Counter .
2. Computes the next sequential address (PC + 4).
3. Selects between PC + 4 and a branch/jump target address.
4. Fetches the 32-bit instruction from instruction memory.
5. Outputs the fetched instruction and PC values for the next stage.

The Program Counter holds the address of the current instruction being fetched. On reset, the PC is cleared to 0. On every rising edge of the clock if en = 1, the PC updates to the next selected address else PC stalls. The next PC value is selected using a control signal PCSrcE. If PCSrcE = 0 sequential execution (PC + 4) takes place, else branch or jump is taken. RISC-V RV32I instructions are 32 bits wide. Hence, the next sequential instruction address is computed as: PCPlus4F = PCF + 4.

The Instruction Memory stores the program instructions in hexadecimal format. It is addressed using PC[31:2] to ensure word-aligned access.The memory provides a combinational read, meaning the instruction is available immediately for a given PC.
Instructions are loaded using $readmemh from an external memory file. The output of the instruction memory is a 32-bit raw instruction, which is passed unchanged to the next pipeline stage.The IF stage produces the following outputs:

- PCF – current program counter value
- PCPlus4F – sequential next address
- InstrF – fetched 32-bit instruction

2. Instruction Decode (ID) Cycle

Name: Raj Kumar Pirangi
Roll Number: 24210079
Assigned Module: Instruction Decode (ID) Cycle

Module Description:

The Instruction Decode (ID) stage is the second stage in a pipelined RISC-V processor and follows the Instruction Fetch (IF) stage. While the fetch stage is responsible for bringing the 32-bit instruction from memory, the decode stage is where the processor begins to understand what the instruction actually means. In this stage, the raw instruction bits are interpreted and converted into control signals, operands, and immediate values that will guide the behavior of the subsequent pipeline stages.

In the given design, the Decode_Cycle module implements the complete functionality of the ID stage. It receives the fetched instruction (InstrD), the current program counter (PCD), and the next sequential PC (PCPlus4D) from the fetch stage. It also interfaces with the writeback stage through RegWriteW, ResultW, and RDW to support register updates. The outputs of this stage form a complete set of data and control signals required by the Execute (EX) stage in the next cycle.

The decode stage performs four main tasks in parallel. First, it extracts instruction fields such as the opcode, source registers, destination register, and function bits from the instruction. These fields determine which registers are read and which register will be written later. Second, the control unit decodes the opcode and function fields to generate control signals that describe the required operation, such as whether the instruction is a load, store, arithmetic operation, branch, or jump. Third, the register file is accessed to read the actual operand values corresponding to the source registers. This is the first point where the instruction interacts with real program data. Finally, the immediate value is reconstructed and sign-extended according to the instruction format, allowing constants and offsets to be used correctly in later stages.

Once decoding is complete, all relevant data and control information are stored in pipeline registers between the ID and EX stages. These registers hold the operand values, immediate, program counter values, destination register number, and control signals. On reset or pipeline clear, these registers are flushed to prevent invalid instructions from propagating further. In the next cycle, the Execute stage receives a fully prepared set of inputs and can focus only on computation and address generation.

Conceptually, the Instruction Decode stage acts as the bridge between instruction representation and instruction execution. Before this stage, the instruction is merely a 32-bit pattern fetched from memory. After decode, the processor has identified the instruction's intent, gathered the required operands, and prepared all control information needed for execution. Although no actual computation is performed in this stage, the correctness and efficiency of the entire pipeline heavily depend on accurate decoding.

3. Register File & Immediate Generation

Name: Akshat Yati
Roll Number: 25210011
Assigned Module: Register File & Immediate Generation

Module Description:

The Register File is a fundamental building block of the RISC-V processor, serving as its rapid-access data storage. Located in the Decode Stage, it holds the processor's architectural state—the values of all 32 general-purpose registers (x0 through x31). Its primary function is to supply the operands required by instructions and to capture the results produced by the ALU or memory.

In this pipelined design, the Register File is a synchronous write, asynchronous read memory. This means data is written on the rising edge of the clock, while reads happen continuously, providing output within the same clock cycle they are requested. This is crucial for the Decode stage to quickly obtain operands for the Execute stage.

Details

- Read Ports (A1, A2): Two dedicated read address ports (A1 and A2) allow the Register File to output the contents of two different registers simultaneously. These outputs,RD1 and RD2, correspond to the source registers specified in an instruction .
- Write Port (A3, WD3): A single write port is used to update a register. During the Write Back stage, the result to be stored(ResultW) is placed on the write-data bus (Wd3), and the destination register address (A3), which comes from the Rd field of the instruction, is used to select which register to update.
- Write Enable (WE3): The RegWriteW signal, generated by the Control Unit and pipelined all the way to the Write Back stage, acts as the write enable (WE3). A write to the register file only occurs if WE3 is high and the destination register is not x0 (which is hardwired to zero).
- x0 Hardwiring: Register x0 is permanently tied to ground, ensuring that any read from it returns 32’d0 and any write to it is ignored. This is a key part of the RISC-V ISA.
- Reset Behavior: When the rst signal is asserted, the read outputs(RD1,RD2)testbench) are forced to zero , preventing any misinform data propagating through the pipeline during reset.

Example:

Consider the instructions add x9,x5,x6 (assembly add x9,x5,x6). In the Decode stage:

- The instruction's rs1 field (bits 19:15) points to x5, which is fed to A1.
- The instruction's rs2 field (bits 24:20) points to x6, which is fed to A2.
- The Register File asynchronously outputs the values stored in x5 and x6 RD1,)and RD2 respectively. These values are then passed down the pipeline to the Execute stage.
- Later, when the instruction completes the Memory stage and reaches Write Back, the ResultW (the sum of x5 and x6) is written into register x9 on the next positive clock edge, as the Rd field (x9) is on A3and RegWriteW is asserted.

4. Immediate Generation

RISC-V instructions encode immediate values in several different formats depending on the instruction type (I, S, B, U, J). The Immediate Generation unit, also located in the Decode stage, is responsible for extracting these variably placed immediate bits from the instruction and extending them to a full 32-bit value that the ALU or other datapath elements can use. This process is known as immediate extension.

The Sign_Extend module performs this task. Its operation is directed by the ImmSrc control signal, which is generated by the Main Decoder based on the instruction's opcode. ImmSrc The signal informs the extension unit which of the five immediate formats to use.

4. Control Unit & Hazard Unit

Name: Rohit Vijay Gupta
Roll Number: 25250033
Assigned Module: Control Unit & Hazard Unit

Module Description:

1. Control Unit

The Control Unit (CU) is responsible for decoding instructions and generating the necessary control signals required for correct operation of the 5-stage pipelined RISC-V processor. In a pipelined architecture, multiple instructions are processed simultaneously across five stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). The Control Unit ensures that each instruction flows correctly through these stages.

The Control Unit primarily operates during the Instruction Decode (ID) stage. It examines the instruction fields such as opcode, funct3, and funct7, and determines the type of instruction (R-type, I-type, Load, Store, Branch, or Jump). Based on this decoding, it generates control signals that guide datapath components including the ALU, register file, memory, and multiplexers.

The main control signals generated include:

- **RegWrite** – Enables writing to the register file
- **ALUSrc** – Selects between register operand or immediate value
- **ALUOp** – Specifies the operation to be performed by the ALU
- **MemRead / MemWrite** – Control memory access
- **Branch** – Indicates branch instructions
- **Jump**– Indicates jump instructions
- **ResultSrc** – Selects the data source for write-back

These control signals are passed through pipeline registers to ensure proper timing and synchronization across stages.

Example

For the instruction:

ADDI x5, x1, 10

The Control Unit generates:

- RegWrite = 1
- ALUSrc = 1
- ALUOp = ADD
- MemRead = 0
- MemWrite = 0

- Branch = 0
- Jump = 0

This ensures that the ALU adds register x1 with immediate value 10 and writes the result into register x5.
Thus, the Control Unit plays a vital role in directing the overall functionality of the processor by controlling data flow and execution logic.

2. Hazard Unit

In a pipelined processor, multiple instructions overlap in execution. This overlapping may cause hazards, which can lead to incorrect results if not handled properly. The Hazard Unit is designed to detect and resolve such hazards to maintain correct program execution.
The Hazard Unit mainly handles:

1. Data Hazards – Occur when an instruction depends on the result of a previous instruction.
2. Control Hazards – Occur due to branch or jump instructions that change program flow.

To resolve these hazards, the Hazard Unit implements the following techniques:

2.1 Data Forwarding (Bypassing)

If an instruction in the Execute stage requires data that has been computed but not yet written back, the Hazard Unit forwards the result directly from later pipeline stages (EX/MEM or MEM/WB) to the ALU inputs. This avoids unnecessary pipeline stalls and improves performance.
Example:

```
add x4, x1, x2
sub x5, x4, x3
```

Here, the second instruction needs the result of the first instruction. Instead of stalling, the Hazard Unit forwards the result directly to the ALU.

2.2 Load-Use Hazard Detection

In case of a load instruction followed immediately by a dependent instruction:

```
lw  x4, 0(x1)
add x5, x4, x6
```

The loaded data is not immediately available. The Hazard Unit detects this dependency and introduces a stall to prevent incorrect data usage.

2.3 Stall and Flush Control

The Hazard Unit generates stall signals to pause the Fetch and Decode stages when required. It also generates flush signals to remove incorrect instructions from the pipeline during branch mispredictions.

5. Main Decoder & Forwarding Logic

Name: Kalaparthi Vikas
Roll Number: 25210064
Assigned Module: Main Decoder & Forwarding Logic

Module Description:

1. Main_decoder

The main decoder in the 5-stage pipelined RISC-V processor operates in the Instruction Decode (ID) stage and is responsible for generating the control signals required for proper datapath operation. The 32-bit instruction is decoded into its constituent fields such as opcode, funct3, funct7, rs1, rs2, and rd. Based on the opcode and function bits, the decoder produces control signals including RegWrite, MemRead, MemWrite, ALUSrc, Branch, and ALUOp. These signals determine whether the ALU uses a register or immediate operand, whether data memory is accessed, and whether the result is written back to the register file. The generated control signals are stored in the ID/EX pipeline register so that they propagate along with the instruction through the EX, MEM, and WB stages, ensuring synchronized execution across the pipeline.

Control Signal Packing

All control signals are grouped into a single 14-bit register called controls, arranged as:

RegWrite_ImmSrc_ALUSrcA_ALUSrcB_MemWrite_ResultSrc_Branch_ALUOp_Jump

Example code: **7'b0000011: controls = 14'b1_000_0_01_0_01_0_00_0; // lw**

This packed format allows each opcode to be represented by one 14-bit control word, simplifying the decoding logic. The assign statement then separates these bits into their respective output control signals.

Output Signals

- **RegWrite :** Enables writing to the register file
- **ImmSrc[2:0] :** Selects immediate format (I, S, B, U, J types)
- **ALUSrcA :** Selects ALU input A (Register or PC)
- **ALUSrcB[1:0] :** Selects ALU input B (Register, Immediate, etc.)
- **MemWrite :** Enables writing to data memory
- **ResultSrc[1:0] :** Selects data to write back (ALU result, memory data, PC+4)
- **Branch :** Indicates a conditional branch instruction
- **ALUOp[1:0] :** Determines ALU operation category
- **Jump :** Indicates an unconditional jump instruction

2. Forwarding logic

The forwarding logic is implemented to resolve data hazards and maintain pipeline efficiency. It operates primarily in the Execute (EX) stage by comparing the source registers of the current instruction in the ID/EX register with the destination registers of instructions in the EX/MEM and MEM/WB stages. If a match is detected and RegWrite is asserted, multiplexers select the forwarded data from a later pipeline stage instead of the register file output. This bypass mechanism allows dependent instructions to use the most recent result without waiting for it to be written back. In cases such as load-use hazards where the required data is not yet available for forwarding, the hazard detection unit introduces a stall to preserve correct execution. This combination of forwarding and stalling ensures both correctness and improved pipeline performance.

6. Execute (EX) Cycle

Name: Amit Kumar
Roll Number: 25210015
Assigned Module: Execute (EX) Cycle

Module Description:

1. Execute (Ex) Cycle:

The Execute Cycle is the third stage in the five-stage RISC-V pipeline, positioned between the Decode stage and the Memory stage. This is where the actual computation happens based on the instruction decoded in the previous stage. The Execute stage takes operands from the Decode stage, performs arithmetic or logical operations, and determines whether branch conditions are met.

Main Components in Execute Stage

The Execute stage contains several key components working together:
ALU (Arithmetic Logic Unit) - Performs all arithmetic and logical operations such as addition, subtraction, AND, OR, XOR, shifts, and comparisons.
PC Adder for Branch Target - Calculates the target address for branch and jump instructions by adding the program counter value with the extended immediate.
Multiplexers - Three 3x1 multiplexers select the correct operands for the ALU, handling both immediate values and forwarded data from later stages.
Pipeline Register - Stores the results and control signals to be passed to the Memory stage.

Operand Selection and Forwarding

Before the ALU can perform its operation, the correct operands must be selected. The Execute stage receives:

- RD1E and RD2E - Register values from the Decode stage
- ImmExtE - The extended immediate value
- PCE - The program counter value for the current instruction
- PCPlus4E- PC + 4 for calculating return addresses
- ResultW - Result from Write Back stage (for forwarding)
- AluResultM - Result from Memory stage (for forwarding)

The Forwarding Unit sends ForwardAE and ForwardBE signals to the three 3x1 multiplexers. These signals determine whether the ALU inputs should come from the register file directly or from forwarded results of later instructions. This resolves data hazards without stalling the pipeline.

ALUSrc Signals and Operand Selection

Two control signals determine the final ALU inputs:
ALUSrcAE - Controls the first ALU input multiplexer:

- When 0: Selects the forwarded register value (or PC for certain instructions)
- When 1: Selects 0 (used for certain operations)

ALUSrcBE - Controls the second ALU input multiplexer with three options:

- 00: Selects the forwarded register value from RD2E
- 01: Selects the extended immediate value
- 10: Selects PCPlus4E (for jump and link instructions)
- 11: Selects PCE (for PC-relative addressing)

Branch and Jump Decision

The Execute stage also determines whether a branch should be taken. The ALU produces a Zero output when the result of an operation is zero. For branch instructions like beq, the ALU performs a subtraction between two registers, and if the result is zero, the Zero signal goes high.

7. ALU Decoder

Name: Arjun AV
Roll Number: 25250004
Assigned Module: ALU Decoder

Module Description:
The decoder block works as a translator. It helps the ALU to figure out which operation to be done after fetching the instruction from the memory.

Inputs it takes

- **ALUOp(2) :**comes from the main control unit and gives ideas about the instruction type (load/store, branch, R/I-type). It can be branch instruction, load/store instruction etc.
- **Funct3 (3 bits) :** from the instruction specifies which group of operation to be done.
- **Funct7 (7 bits) :** from the instruction specifies which exact operation to be done.
- **Op (opcode – 7 bits) →:**from the instruction which helps to identify the instruction format/type.

Output it gives

- **ALUControl[3:0]** → this goes to the ALU which selects the exact operation to perform

FLOW OF THE CODE

The ALU decoder continuously monitors its inputs and first evaluates ALUOp to determine whether the instruction is a load/store, branch, or R/I-type. For ALUOp = 00 it directly selects addition and for ALUOp = 01 it selects subtraction without checking any other fields. When ALUOp indicates an R-type or I-type instruction, the decoder examines funct3 to identify the operation group. For funct3 = 000 it distinguishes between ADD and SUB using the RtypeSub signal derived from funct7 and opcode. For shift-right operations (funct3 = 101) it uses funct7 to choose between SRL and SRA. For all other funct3 values the corresponding ALU operation is selected directly. The resulting 4-bit ALUControl signal is then sent to the ALU to perform the required operation.

Case Example

If the instruction is Sub the main control unit sends ALUOp = 10, which tells the ALU decoder that this is an R-type ALU operation and that it must look at the instruction fields to decide the exact function. The decoder then checks funct3 = 000, which represents the ADD/SUB group, so at this stage it still cannot distinguish between addition and subtraction. To make that distinction it generates the internal signal RtypeSub = funct7[5] & op[5]. For an R-type SUB instruction the opcode corresponds to the R-type format (so op[5] = 1) and the special SUB encoding in funct7 has bit 5 equal to 1, whereas for ADD this bit would be 0. Because both bits are 1 only for the R-type SUB case, RtypeSub becomes 1, and the decoder selects ALUControl = 0001, which tells the ALU to perform subtraction on the values in x6 and x7. If it had been an ADD or ADDI instruction, funct7[5] or op[5] would be 0, making RtypeSub = 0, and the decoder would instead output the control for addition

8. Memory Access (MEM) Cycle

Name: Prem Singh Bilavth
Roll Number: 25210098
Assigned Module: Memory Access (MEM) Cycle

Module Description:Data Memory

The Data_memory module is a 1024 × 32-bit memory block used in RISC-V architecture to perform load and store operations. It supports synchronous write operations controlled by a write enable signal and asynchronous read operations controlled by reset. During store instructions, data is written into memory at the calculated address. During load instructions, data is read from memory and sent to the register file. The module also includes a memory dump feature for simulation and debugging purposes. It plays a critical role in the MEM stage of the RISC-V pipeline and is essential for implementing the load-store architecture of RISC-V processors.

Input Pins Description

1. clk (Clock Input)

Direction: Input
Width: 1 bit

The clock signal controls the synchronous write operation of the memory. Data is written into memory only on the rising edge (positive edge) of the clock when the Write Enable (WE) signal is active. This ensures proper synchronization with the processor clock.

2. rst (Reset Input)

Direction: Input
Width: 1 bit
The reset signal controls the read data output. When rst = 0, the output data (RD) is forced to 0. When rst = 1, normal memory read operation takes place. This signal ensures stable and defined output during initialization.

3. WE (Write Enable)

Direction: Input
Width: 1 bit
The Write Enable signal controls whether data is written into memory or not.

- If WE = 1, data is written into the memory at the specified address.
- If WE = 0, no write operation occurs.

This signal is activated during store instructions in RISC-V architecture.

4. A [31:0] (Address Input)
Direction: Input
Width: 32 bits
The address input specifies the memory location to be accessed for read or write operations. It selects one of the 1024 memory locations available in the memory array. In RISC-V architecture, this address is typically generated by the ALU during load and store instructions.

5. WD [31:0] (Write Data Input)
Direction: Input
Width: 32 bits
The Write Data input carries the 32-bit data value that will be stored in memory at the specified address when the Write Enable signal is active. This data usually comes from the register file during store operations.

Output Pin Description

1. RD [31:0] (Read Data Output)
Direction: Output
Width: 32 bits
The Read Data output provides the 32-bit data stored at the memory location specified by the address input. If the reset signal is low (rst = 0), the output is forced to zero. Otherwise, the selected memory data is available at the output. This output is used during load instruction transfer data back to the register file.

Memory_Cycle Module

The Memory_Cycle module implements the Memory (MEM) and Write-Back (WB) stages of a pipelined RISC-V processor. It instantiates the Data memory block to perform load and store operations using the ALU result as the memory address. During store instructions, data is written into memory, and during load instructions, data is read from memory.
The module also contains pipeline registers that transfer memory data, ALU results, destination register address, and control signals from the MEM stage to the WB stage. This ensures proper synchronization and correct write-back of results to the register file in a 5-stage RISC-V pipeline.

Inputs

- **clk (1 bit)** – Clock signal for updating pipeline registers.
- **rst (1 bit)** – Active-low reset signal to clear outputs.
- **RegWriteM (1 bit)** – Register write control from MEM stage.
- **MemWriteM (1 bit)** – Memory write enable signal.
- **ResultSrcM (2 bits)** – Select signal for write-back data source.
- **ALUResultM (32 bits)** – ALU result used as memory address or arithmetic result.
- **WriteDataM (32 bits)** – Data to be written into memory (store operation).
- **RdM (5 bits)** – Destination register number from EX stage.
- **PCPlus4M (32 bits)** – PC + 4 value for jump instructions.

Outputs

- **ReadDataW (32 bits)** – Data read from memory for load instructions.
- **ALUResultW (32 bits)** – ALU result forwarded to WB stage.
- **PCPlus4W (32 bits)** – PC + 4 forwarded to WB stage.
- **RegWriteW (1 bit)** – Register write control for WB stage.
- **ResultSrcW (2 bits)** – Write-back source select signal.
- **RdW (5 bits)** – Destination register number for WB stage.

9. Pipeline Registers (IF/ID, ID/EX, EX/MEM, MEM/WB)

Name: Oruganti Dheekshitha
Roll Number: 22110176
Assigned Module: Pipeline Registers

Module Description:
The Pipeline Registers are the fundamental synchronization components that enable the 5-stage pipelined execution of the RISC-V processor. Their primary function is to act as buffers between successive stages—Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB)—storing the output of one stage and providing it as a stable input to the next on the rising edge of the clock.

The design incorporates four distinct sets of registers to ensure correct instruction flow:

- **IF/ID Register:** This register receives the 32-bit instruction (InstrF) and the current Program Counter (PCF) from the Fetch stage. It provides these values to the Decode stage (InstrD, PCD) so the next instruction can be fetched simultaneously.

- **ID/EX Register:** It stores decoded operand values from the register file (RD1, RD2), the sign-extended immediate (ImmExt), and all control signals (such as RegWrite, ALUSrc, and ALUControl) required for the Execution stage.

- **EX/MEM Register:** This register carries the computed ALUResult, the data to be written to memory (WriteData), and the destination register address (Rd) forward to the Memory Access stage.

- **MEM/WB Register:** It holds the final data retrieved from memory (ReadData) and the ALUResult, allowing the Write Back cycle to select the appropriate result for the register file.

  Control and Hazard Logic Integration: Beyond data transfer, these registers are crucial for handling pipeline hazards through the following mechanisms:

1. **Stalling (Enable Signal):** Controlled by the Hazard Unit, the en signal can pause specific registers (like IF/ID and PC). When a load-use hazard occurs, the register

maintains its current value, preventing new instructions from entering the pipeline until the hazard is resolved.

2. **Flushing (Clear Signal):** Controlled by the clr or flush signal, these registers can be reset to zero . This is used during branch mispredictions to remove incorrect instructions from the pipeline, effectively inserting "NOP" (No-Operation) cycles to maintain architectural integrity.

10. Top-Level Integration

Name: Arunteja Palutla
Roll Number: 24210017
Assigned Module: Top-Level Integration

Module Description:
A cycle-accurate, 5-stage pipelined RISC processor core implemented in Verilog. This top-level module integrates Fetch, Decode, Execute, Memory, and Writeback stages into a unified datapath. It features a robust Hazard Unit that manages data and control hazards through stalling, flushing, and forwarding logic, ensuring high-throughput execution while maintaining architectural integrity.

The 5 Stages:

- **Fetch (Fetch_Cycle):** Retrieves the instruction from memory and increments the Program Counter (PC).
- **Decode (Decode_Cycle):** Translates the instruction, reads values from the Register File, and extends immediate values.
- **Execute (Execute_Cycle):** Performs arithmetic/logical operations via the ALU and calculates branch targets.
- **Memory (Memory_Cycle)**: Reads from or writes data to the Data Memory.
- **Writeback (Writeback_Cycle):** Selects the final result (from ALU or Memory) and writes it back to the destination register.

The Control & Hazard Logic:

- **Hazard_Unit:** This is the "brain" for stability. It detects Data Hazards (when an instruction needs a result not yet written) and Control Hazards (when a branch is taken). It handles these by injecting Stalls (pausing stages) or Flushes (clearing incorrect instructions).
- **Interim Wires:** These are the physical "wires" that carry data (like ALUResult) and control signals (like RegWrite) across the pipeline registers**.**